PUCP

PROGRAMACIÓN 3 (1INF30)



Acceso a Datos con C#

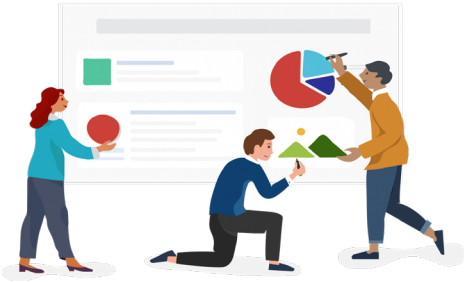
2026

Profesores:

- Dr. Freddy Paz
- Dr. José Luis Corcuera
- Mag. Eric Huiza
- Mag. David Allasi

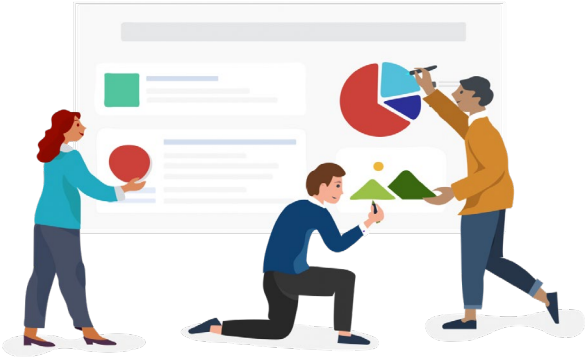
AGENDA

1. Visual Studio
2. Estructura Básica de Visual Studio
3. Visual Studio Code
4. Arquitectura en Capas
5. Instalación de Visual Studio
6. Capa de Dominio
7. Capas en Visual Studio
8. Proyecto SoftProgDBManager
9. ADO .NET
10. Configuración con appsettings.json
11. Clase DBManager
12. Instalación de librerías en C#



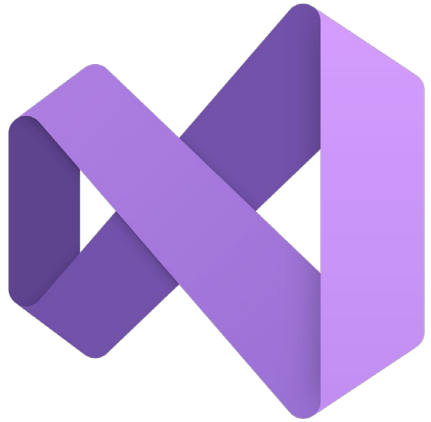
AGENDA

13. Creación del Proyecto Consola SoftProg
14. Creación de appsettings.json
15. Referencias entre Proyectos en la Solución
16. Inserción de Datos desde el Método Main()



¿Qué es Visual Studio?

- **Visual Studio** es un **entorno de desarrollo integrado** (**IDE**) que permite a los desarrolladores escribir, organizar, compilar, ejecutar y depurar aplicaciones en múltiples lenguajes de programación, tales como **C#, C++, Visual Basic .NET, F#, JavaScript y TypeScript**, dentro del ecosistema de desarrollo de Microsoft.



- Este entorno proporciona un conjunto de herramientas que facilitan el desarrollo de software, incluyendo autocompletado de código (**IntelliSense**), detección de errores en tiempo real, administración de proyectos y depuración paso a paso de las aplicaciones.

Estructura Básica de Visual Studio

- El Explorador de soluciones (**Solution Explorer**) permite visualizar y gestionar la estructura del proyecto, mostrando la organización de archivos, clases y configuraciones dentro de la solución.
- El editor de código es el espacio en el que se desarrolla la lógica del programa, brindando soporte mediante resaltado de sintaxis y sugerencias automáticas para facilitar la escritura del código.
- Las opciones de ejecución del programa permiten compilar y correr la aplicación, ya sea en modo de depuración o ejecución directa, facilitando la validación del comportamiento del sistema.
- La consola de salida (Output) muestra los resultados de la ejecución y los posibles errores, permitiendo identificar y corregir problemas durante el desarrollo.
- En este contexto, es importante comprender que una solución puede contener uno o varios proyectos, y que cada proyecto representa una unidad funcional de desarrollo dentro del sistema.

¿Qué es Visual Studio Code?

- **Visual Studio Code** es un editor de código fuente ligero y multiplataforma, diseñado para el desarrollo de aplicaciones en una amplia variedad de lenguajes de programación, tales como **JavaScript, TypeScript, Python, C#, Java, C++ y Go, entre otros**.

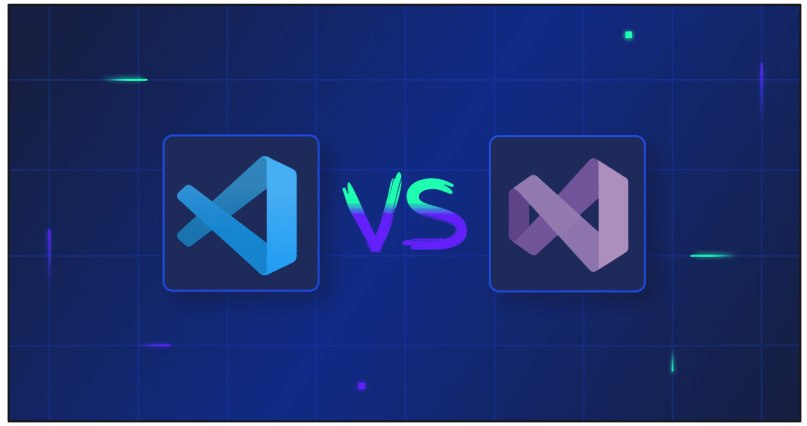


- A diferencia de **Visual Studio**, **Visual Studio Code NO es un entorno de desarrollo integrado completo por defecto**, sino que su funcionalidad puede ampliarse mediante el uso de extensiones, las cuales permiten incorporar herramientas de depuración, control de versiones, soporte para ***frameworks*** y conexión con distintos entornos de ejecución.

¿Qué es Visual Studio Code?

- Este editor se caracteriza por su rapidez, flexibilidad y bajo consumo de recursos, lo que lo convierte en una herramienta ampliamente utilizada tanto en entornos académicos como profesionales.

- En el contexto del curso, **Visual Studio Code** puede utilizarse como alternativa para la edición de código.



Arquitectura en Capas

- La arquitectura en capas es un enfoque de diseño que permite organizar un sistema en diferentes niveles, donde cada capa tiene una responsabilidad específica y se comunica con las capas adyacentes.
- Este enfoque facilita la modularidad, mantenibilidad y escalabilidad del sistema, permitiendo que los cambios en una capa no afecten directamente a las demás.
- En el contexto del curso, trabajaremos con una solución denominada **SoftProgSolution**, la cual estará organizada en múltiples proyectos que representan las distintas capas del sistema.

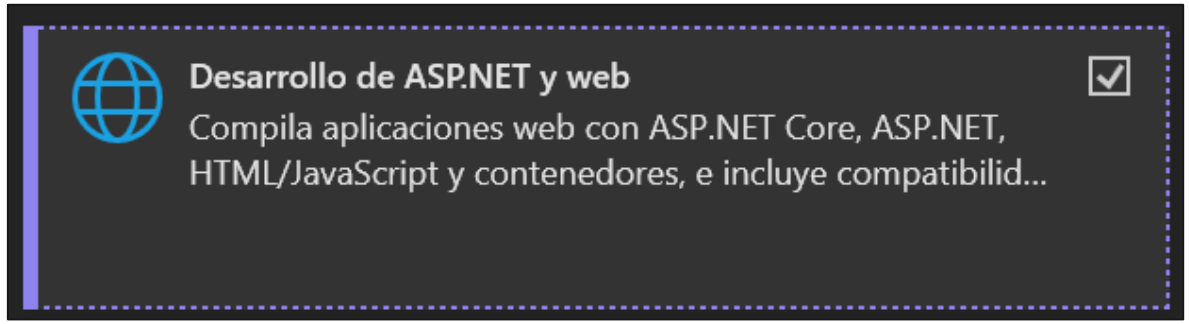


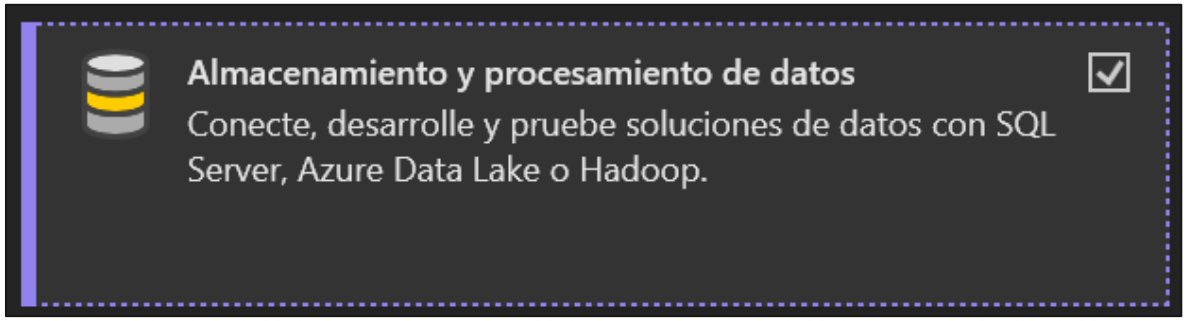
Instalación de Visual Studio

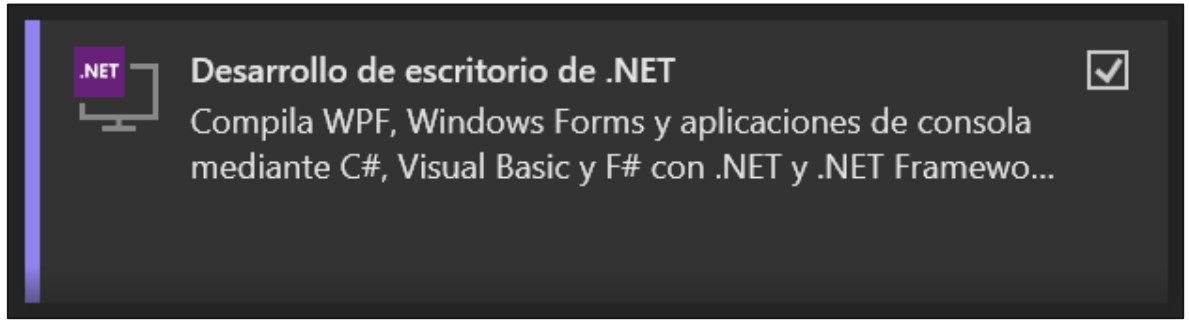
- Para el desarrollo de las actividades, es necesario instalar **Visual Studio** con los componentes adecuados que permitan trabajar con el **lenguaje C#** y el **entorno .NET**, incluyendo el desarrollo de **aplicaciones web** con **Blazor**.
- Durante el proceso de instalación, se deben seleccionar las siguientes cargas de trabajo (***workloads***):
  - **Desarrollo de escritorio con .NET**, el cual permite la creación y ejecución de aplicaciones en C#, incluyendo proyectos de consola que serán utilizados en el curso.
  - **Desarrollo de ASP.NET y web**, necesario para la construcción de aplicaciones web, incluyendo el uso de **Blazor** para el desarrollo de interfaces modernas basadas en C#.
  - **Almacenamiento y procesamiento de datos**, que incorpora herramientas para la conexión y gestión de bases de datos desde el entorno de desarrollo.

Instalación de Visual Studio

- Seleccionamos las siguientes opciones:









Capa de Dominio

- La capa de dominio (modelo), es responsable de representar los conceptos del negocio mediante clases que encapsulan atributos y comportamientos relevantes para el sistema.
- En esta capa se definen las entidades del dominio, tales como: Area, Empleado, Cliente, Producto, entre otras, las cuales son utilizadas por la lógica de negocio y el acceso a datos.
- Es importante destacar que estas clases no deben depender de detalles de implementación como la base de datos o la interfaz de usuario, ya que su propósito es modelar el negocio de forma independiente.
- Esta capa será implementada como un proyecto dentro de la solución denominado SoftProgDomain o SoftProgModel, el cual será reutilizado por las demás capas del sistema.

Capa de Dominio

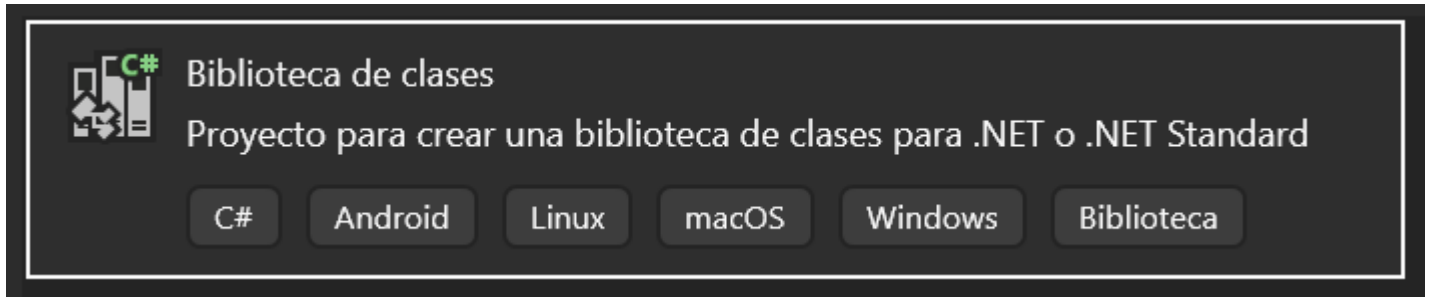
- Estas clases son independientes de otras capas, por lo que no deben contener lógica relacionada con la base de datos, la interfaz de usuario u otros aspectos de infraestructura.
- A diferencia de enfoques centrados en la persistencia, esta capa no representa directamente las tablas de la base de datos ni estructuras de transferencia de datos (DTO), sino el modelo conceptual del negocio.
- Asimismo, las entidades pueden incluir comportamientos y validaciones propias, contribuyendo a mantener la coherencia del dominio. Esta capa será implementada en el proyecto SoftProgDomain o SoftProgModel, el cual será reutilizado por las demás capas del sistema.

Capas en Visual Studio

- Para implementar una arquitectura en capas en Visual Studio, es necesario partir de la creación de una solución, la cual actúa como contenedor de los distintos proyectos que representarán las capas del sistema.
- Una solución permite organizar de manera estructurada los componentes del sistema, facilitando la separación de responsabilidades y la reutilización de código entre las distintas capas.
- Se creará una solución denominada SoftProgSolution, dentro de la cual se incorporarán progresivamente los proyectos correspondientes a cada capa de la arquitectura.
- Como primer paso, se añadirá un proyecto de tipo Class Library denominado SoftProgDomain, el cual representará la capa de modelo o dominio del sistema.
- Este proyecto contendrá las entidades del negocio y será utilizado por las demás capas, constituyendo la base para el desarrollo del sistema.

Capas en Visual Studio

- Identificamos la plantilla a utilizar:

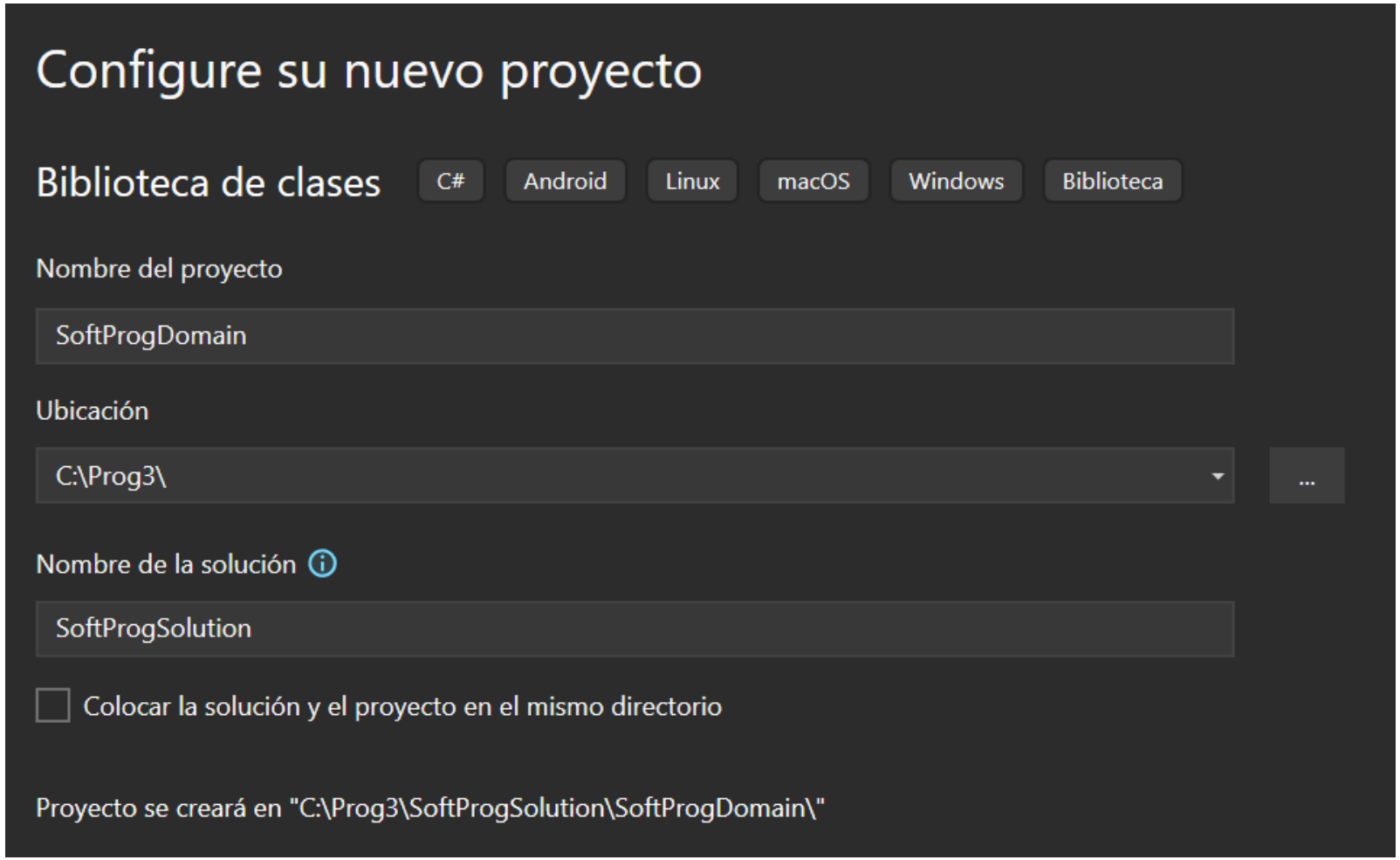


- Esto se debe a que el proyecto SoftProgDomain será de tipo biblioteca de clases (Class Library), ya que su propósito es definir las entidades del dominio y ser reutilizado por las demás capas del sistema.
- Este tipo de proyecto no genera una aplicación ejecutable, sino un conjunto de clases que pueden ser referenciadas por otros proyectos.

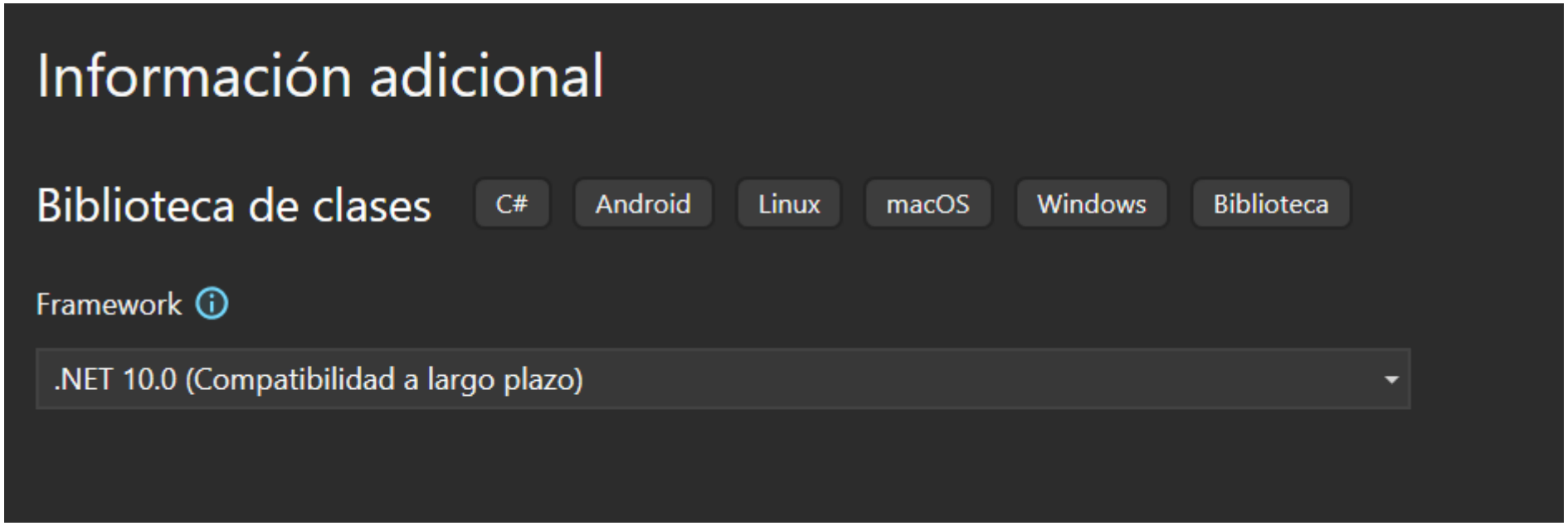
Capas en Visual Studio

- Nombre del proyecto: Se define como SoftProgDomain, siguiendo la convención de nombres en PascalCase y representando la capa de dominio del sistema.
- Nombre de la solución: Se establece como SoftProgSolution, ya que agrupa todos los proyectos que conformarán las distintas capas del sistema.
- Ubicación del proyecto: Permite organizar físicamente los archivos del sistema en el equipo; se recomienda una ruta clara y accesible para el desarrollo.
- Tipo de proyecto (Class Library): Se selecciona este tipo porque el proyecto contendrá clases reutilizables y no una aplicación ejecutable.

Capas en Visual Studio



Capas en Visual Studio



Capas en Visual Studio

- En C#, la organización lógica del código se realiza mediante espacios de nombres (namespaces), los cuales permiten agrupar clases relacionadas dentro del sistema.
- A diferencia de Java, donde se utilizan paquetes con rutas como `pe.edu.pucp.softprog...`, en C# los namespaces se definen a partir del nombre del proyecto y la estructura de carpetas.
- En el proyecto SoftProgDomain, se crearán carpetas que representen los distintos módulos del dominio, tales como: RRHH, GestionClientes, Ventas, Almacen, etc.
- Cada carpeta define automáticamente un namespace. Esta organización permite mantener el código estructurado, facilitando su comprensión, mantenimiento y reutilización dentro del sistema.

Capas en Visual Studio

- A continuación, se muestra un ejemplo:

```
SoftProgDomain
├── RRHH
│   ├── Area.cs
│   ├── Empleado.cs
│   ├── Persona.cs
│   └── CuentaUsuario.cs
├── GestionClientes
│   ├── Cliente.cs
│   └── Categoria.cs
├── Ventas
│   ├── OrdenVenta.cs
│   └── LineaOrdenVenta.cs
├── Almacen
│   └── Producto.cs
```

Capas en Visual Studio

Area.cs | Novedades

SoftProgDomain | SoftProgDomain.RRHH.Area | Nombre

```
1   namespace SoftProgDomain.RRHH
2   {
        0 referencias
3       public class Area
4       {
5           private int _idArea;
6           private String? _nombre;
7           private bool _activa;
            0 referencias
8           public int IdArea { get => _idArea; set => _idArea = value; }
            0 referencias
9           public string? Nombre { get => _nombre; set => _nombre = value; }
            0 referencias
10          public bool Activa { get => _activa; set => _activa = value; }
11      }
12  }
```

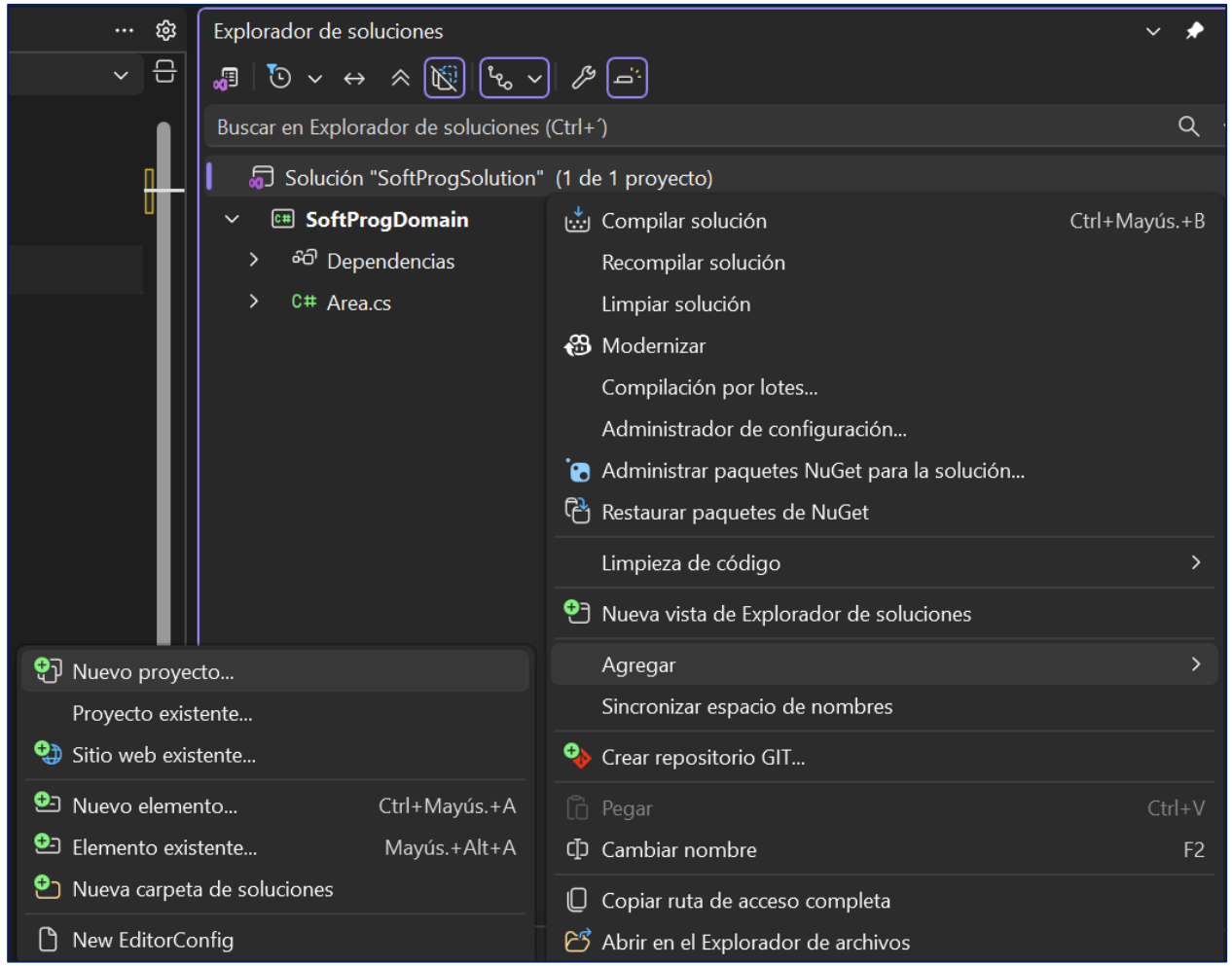
Proyecto SoftProgDBManager

- Para gestionar la conexión a la base de datos y centralizar las operaciones de acceso a datos, se creará un nuevo proyecto dentro de la solución en Visual Studio.
- Este proyecto se denominará SoftProgDBManager y será de tipo Class Library, ya que contendrá clases reutilizables encargadas de la conexión y manejo de la base de datos.
- Su principal responsabilidad será proporcionar los mecanismos necesarios para establecer la conexión con la base de datos, así como servir de base para la implementación de la capa de acceso a datos (DAO).
- Este proyecto dependerá del uso de ADO.NET, junto con un proveedor de datos (por ejemplo, MySQL Connector), para gestionar la conexión a la base de datos, y será utilizado por otras capas del sistema, como la lógica de negocio y el acceso a datos.
- La separación de este proyecto permite mantener una adecuada organización del sistema, promoviendo la reutilización del código y la independencia entre capas.

ADO .NET

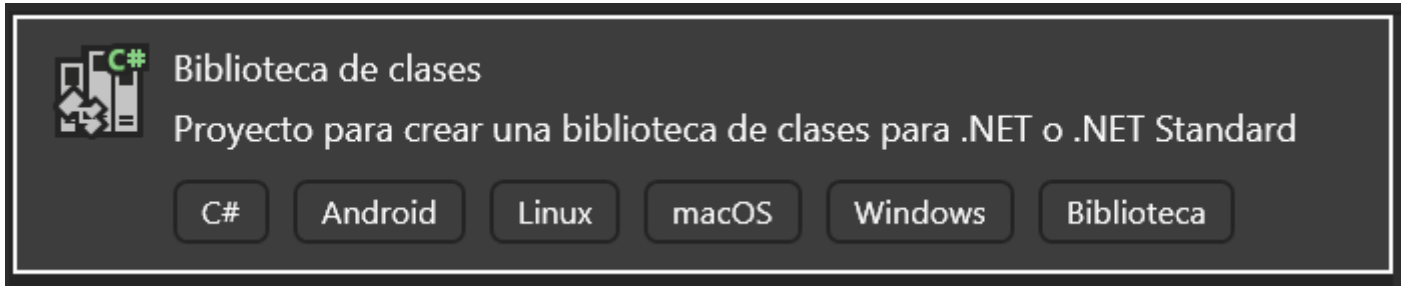
- ADO.NET es una tecnología del entorno .NET que proporciona un conjunto de clases para el acceso a datos desde aplicaciones desarrolladas en C#.
- Permite establecer conexiones con distintos motores de base de datos, ejecutar instrucciones SQL y recuperar información de manera estructurada.
- ADO.NET trabaja a través de proveedores de datos, los cuales son específicos para cada sistema gestor de bases de datos, como SQL Server o MySQL.
- Entre sus principales componentes se encuentran:
  - Connection: establece la conexión con la base de datos.
  - Command: ejecuta consultas.
  - SQLDataReader: permite leer resultados de forma eficiente.
  - DataSet: representa datos en memoria.

Proyecto SoftProgDBManager



- Clic derecho a la Solución SoftProgSolution y luego “Agregar”, “Nuevo Proyecto”.

Proyecto SoftProgDBManager



- El proyecto SoftProgDBManager se define como una biblioteca de clases (Class Library), ya que su propósito no es ejecutar una aplicación, sino proveer funcionalidades reutilizables relacionadas con la conexión a la base de datos.
- Este proyecto contendrá clases encargadas de establecer la conexión utilizando ADO.NET y un proveedor de datos (por ejemplo, MySQL Connector), las cuales serán utilizadas por otras capas del sistema.
- Al no ser una aplicación ejecutable, este proyecto no posee un punto de entrada (Main), sino que es referenciado por otros proyectos, como la lógica de negocio o la capa de acceso a datos.

Configuración con appsettings.json

- En aplicaciones modernas de .NET, la configuración del sistema se gestiona mediante el archivo appsettings.json, el cual permite definir parámetros como cadenas de conexión, configuraciones de entorno y valores generales de la aplicación.
- Este archivo utiliza formato JSON, lo que facilita la organización y lectura de los datos de configuración.
- A diferencia de enfoques tradicionales como app.config o archivos .properties, appsettings.json es el estándar en aplicaciones basadas en .NET moderno, incluyendo ASP.NET y Blazor.
- En el contexto del curso, se utilizará este archivo para almacenar la cadena de conexión a la base de datos, la cual será consumida por el proyecto SoftProgDBManager.
- El archivo appsettings.json se coloca en el proyecto de inicio (startup project).

Clase DBManager

```
public class DBManager
{
    private static DBManager _instance;
    private readonly string _connectionString;

    public static DBManager Instance
    {
        get { return _instance; }
    }
    private DBManager(string connectionString)
    {
        _connectionString = connectionString;
    }
    public static void Initialize(string connectionString)
    {
        if (_instance == null)
        {
            _instance = new DBManager(connectionString);
        }
    }
    public MySqlConnection GetConnection()
    {
        return new MySqlConnection(_connectionString);
    }
}
```

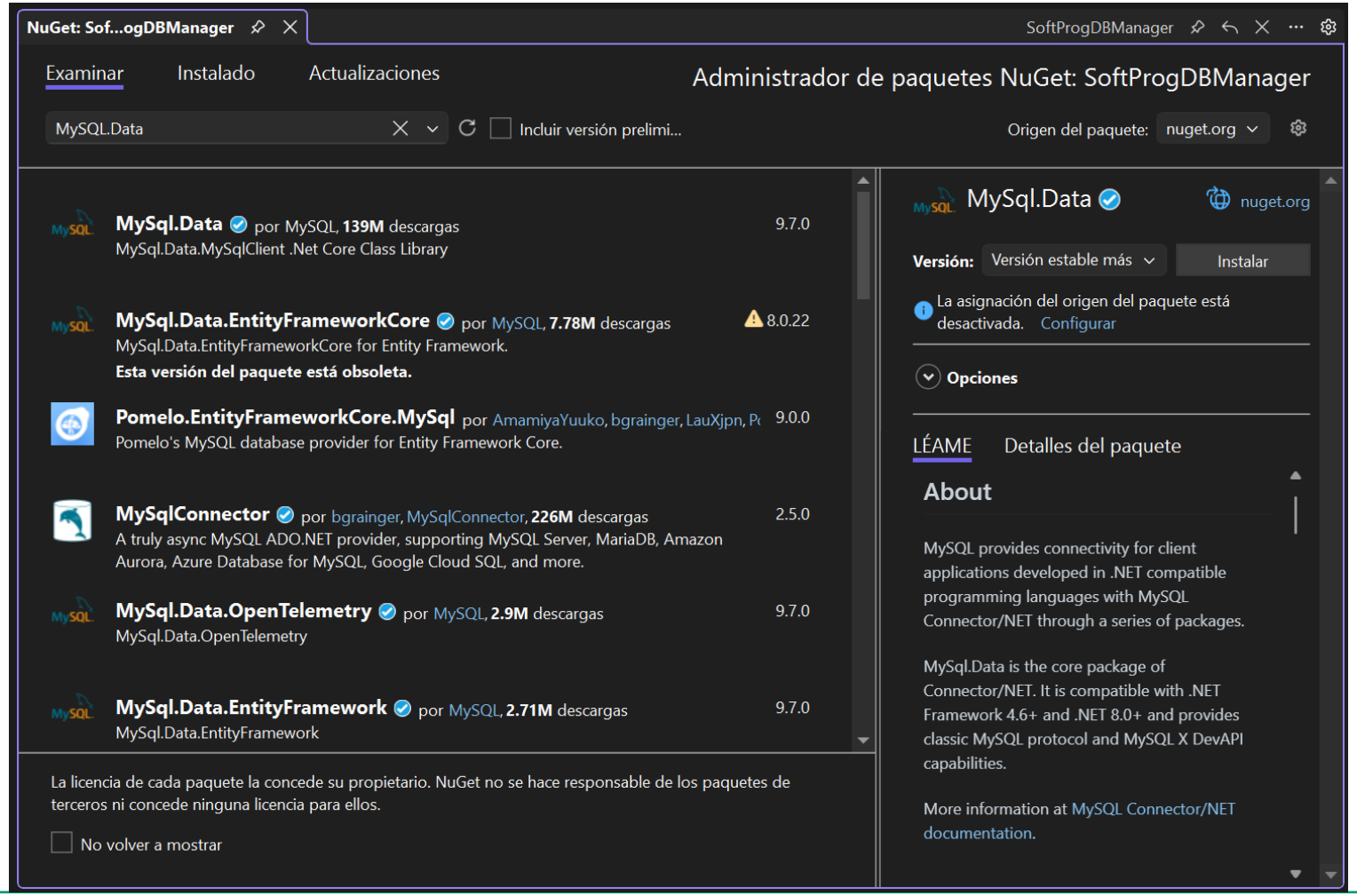
Instalación de librerías en C#

- En aplicaciones desarrolladas en C#, es necesario incorporar librerías externas para extender la funcionalidad del sistema, como el acceso a bases de datos o la gestión de configuración.
- En el entorno de Visual Studio, estas librerías se gestionan mediante NuGet, el administrador de paquetes de .NET.
- La instalación de estos paquetes se realiza a través de la opción “Manage NuGet Packages” del proyecto en Visual Studio.
- De manera alternativa, es posible descargar manualmente los archivos .dll y agregarlos como referencias al proyecto.

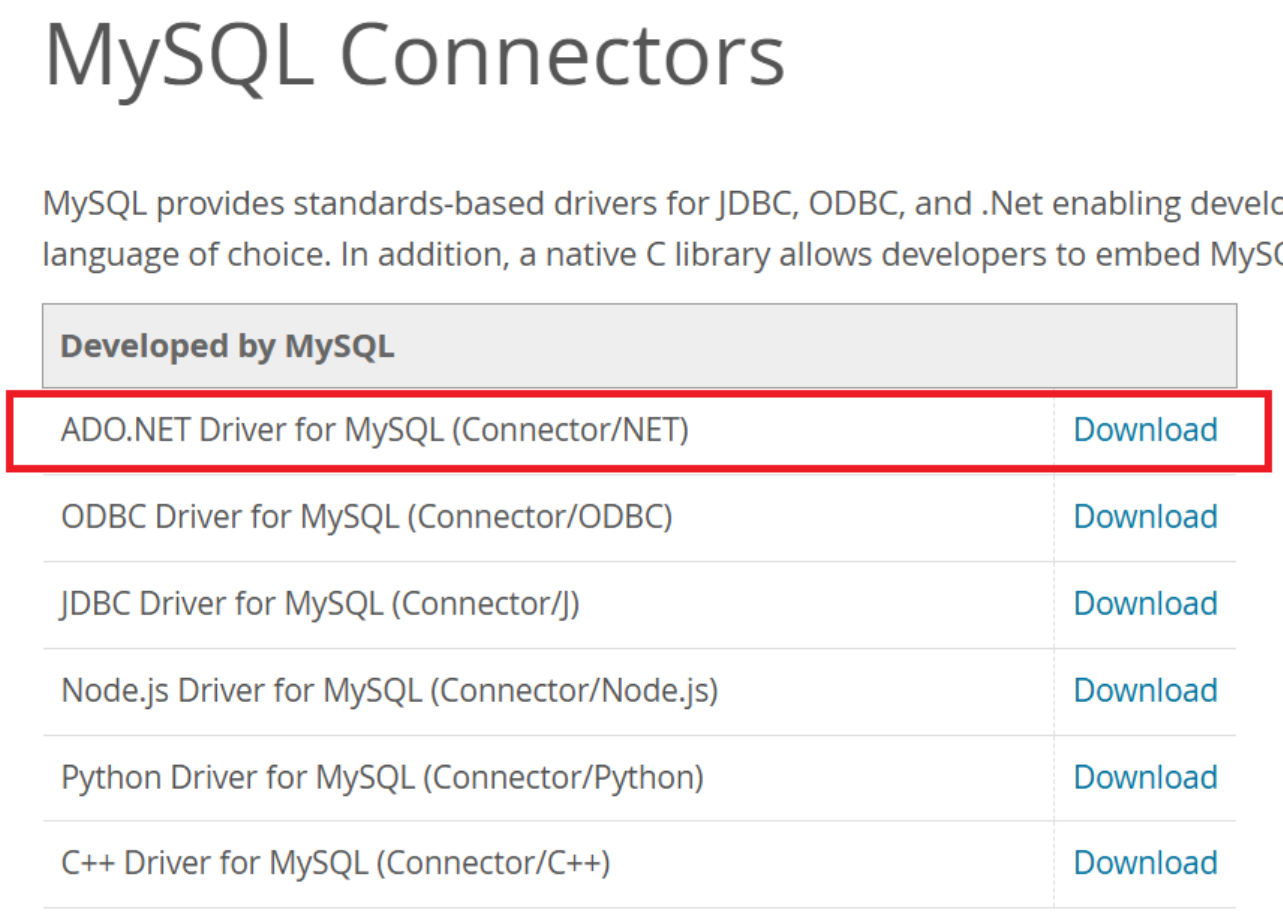
Instalación de librerías en C#

- Para trabajar con bases de datos MySQL, es necesario instalar el paquete:
  - MySql.Data, el cual proporciona clases como MySqlConnection, MySqlCommand y MySqlDataReader.
- Asimismo, para el manejo de archivos de configuración (appsettings.json), se deben instalar los siguientes paquetes:
  - Microsoft.Extensions.Configuration
  - Microsoft.Extensions.Configuration.Json
  - Microsoft.Extensions.Configuration.FileExtensions
  - System.Configuration.ConfigurationManager
  - System.Security.Permissions

Instalación de librerías en C# - Mediante NuGet



Instalación de librerías en C# - Manera Manual



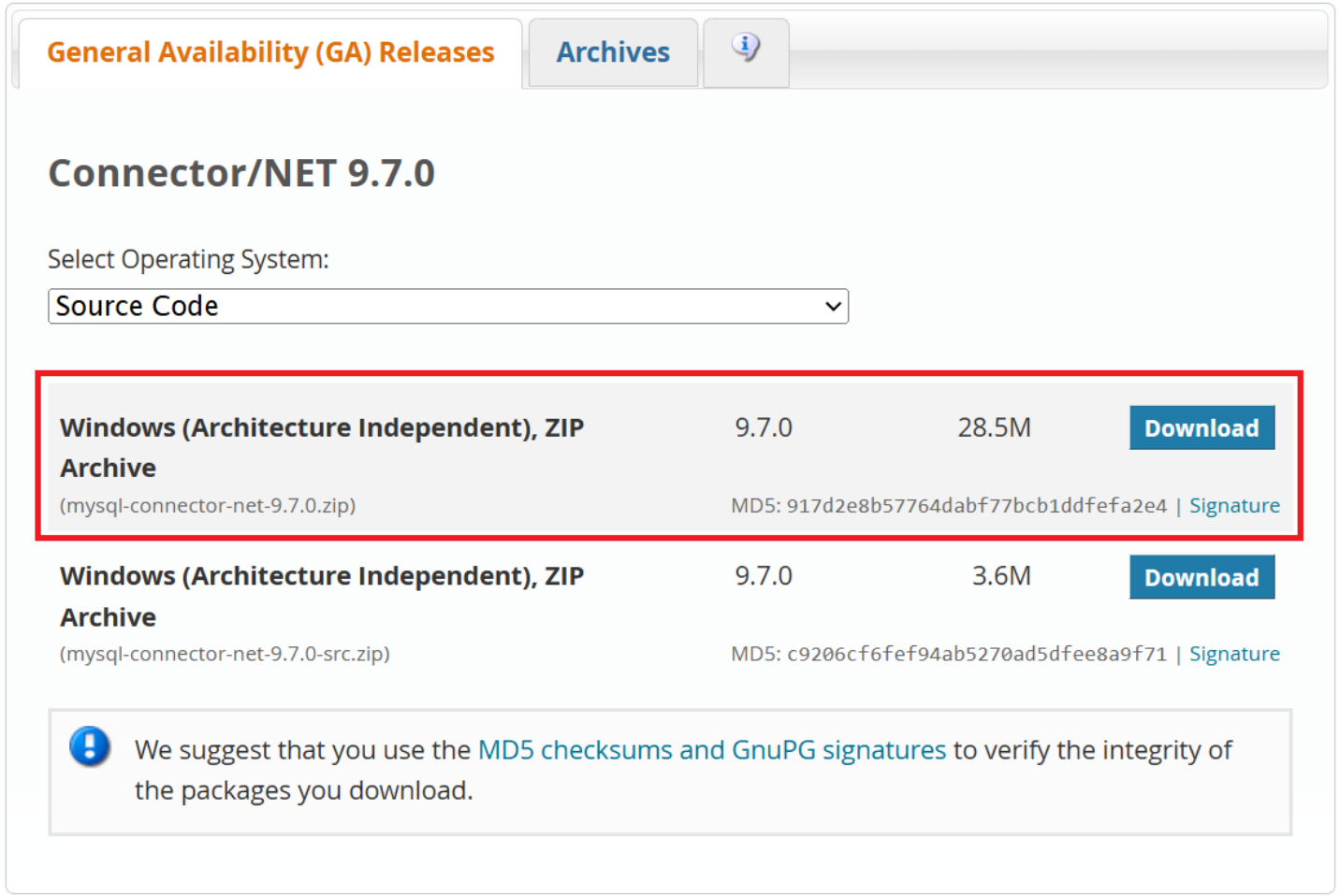
MySQL Connectors

MySQL provides standards-based drivers for JDBC, ODBC, and .Net enabling develope
language of choice. In addition, a native C library allows developers to embed MySQL

Developed by MySQL	
ADO.NET Driver for MySQL (Connector/NET)	Download
ODBC Driver for MySQL (Connector/ODBC)	Download
JDBC Driver for MySQL (Connector/J)	Download
Node.js Driver for MySQL (Connector/Node.js)	Download
Python Driver for MySQL (Connector/Python)	Download
C++ Driver for MySQL (Connector/C++)	Download

- Ingresamos a la página de https://www.mysql.com/products/connector/
- Descargamos ADO.NET Driver for MySQL

Instalación de librerías en C# - Manera Manual

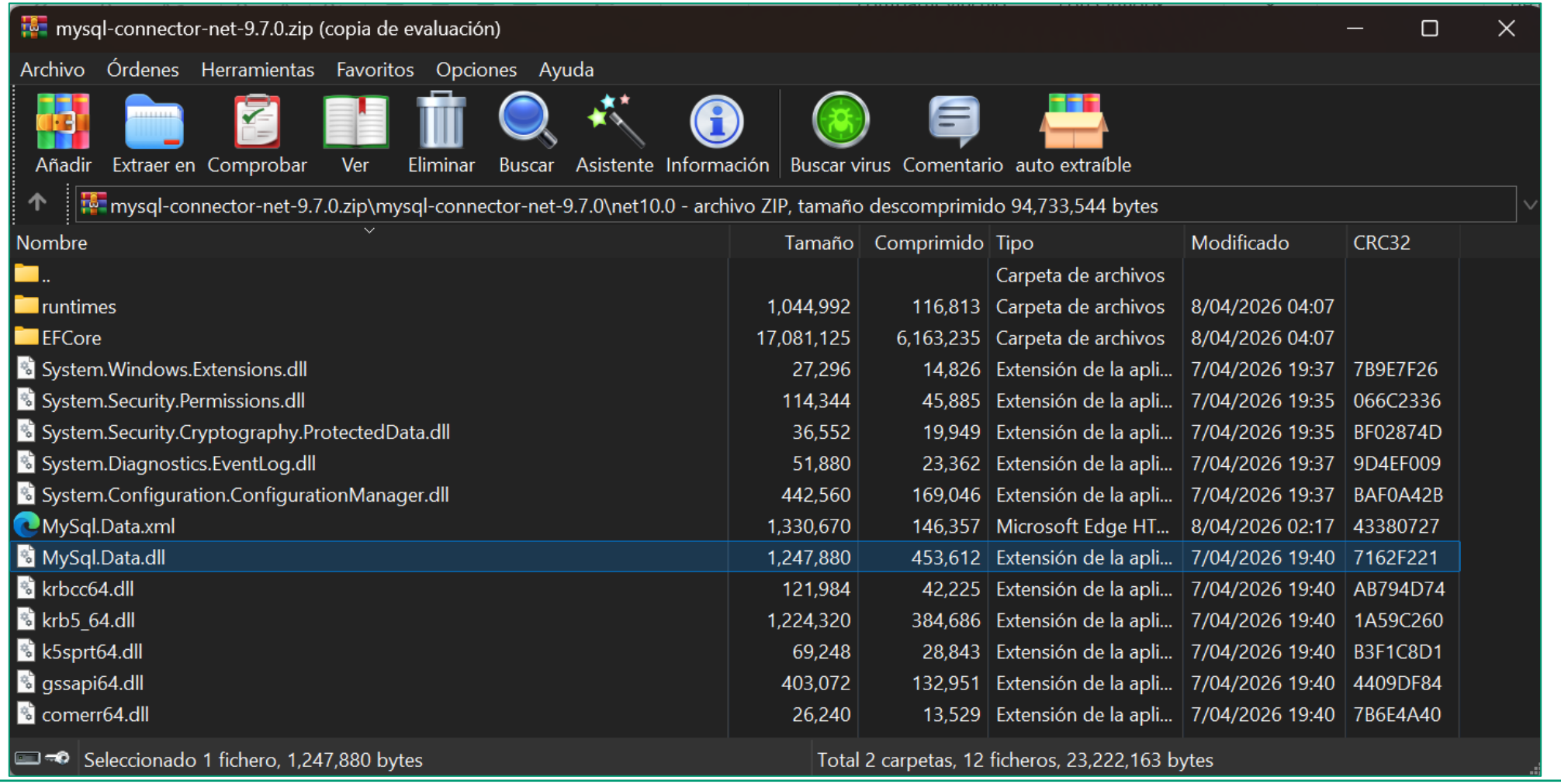


Instalación de librerías en C# - Manera Manual

mysql-connector-net-9.7.0.zip\mysql-connector-net-9.7.0 - archivo ZIP, tamaño descomprimido 94,733,544 bytes

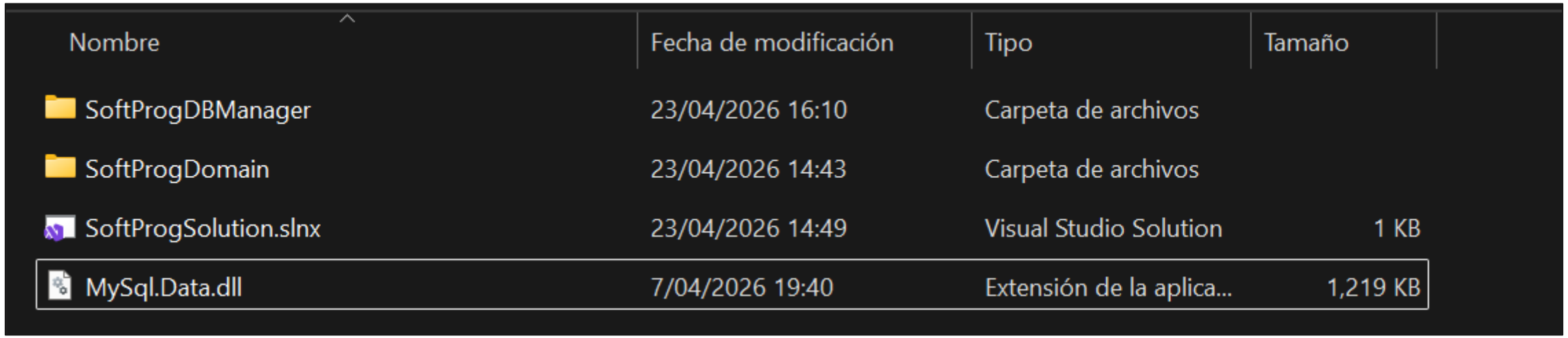
Nombre	Tamaño	Comprimido	Tipo	Modificado	CRC32
..			Carpeta de archivos		
v4.8	10,536,078	3,097,979	Carpeta de archivos	8/04/2026 04:07	
v4.6.2	9,097,146	2,938,974	Carpeta de archivos	8/04/2026 04:07	
netstandard2.1	5,195,222	1,523,213	Carpeta de archivos	8/04/2026 04:07	
netstandard2.0	5,554,105	1,665,503	Carpeta de archivos	8/04/2026 04:07	
net10.0	23,222,163	7,755,319	Carpeta de archivos	8/04/2026 04:07	
net9.0	23,042,490	7,697,830	Carpeta de archivos	8/04/2026 04:07	
net8.0	17,928,526	5,107,859	Carpeta de archivos	8/04/2026 04:07	
Release Notes.txt	452	253	Documento de texto	8/04/2026 02:15	BE07EC04
README	631	346	Archivo	8/04/2026 02:15	37615521
LICENSE	108,080	23,362	Archivo	8/04/2026 02:15	4D5C509E
INFO_SRC	334	174	Archivo	8/04/2026 02:15	943F4062
INFO_BIN	270	154	Archivo	8/04/2026 02:15	13C9ADD6
CHANGES	48,047	14,783	Archivo	8/04/2026 02:15	A8D81A95

Instalación de librerías en C# - Manera Manual



Instalación de librerías en C# - Manera Manual

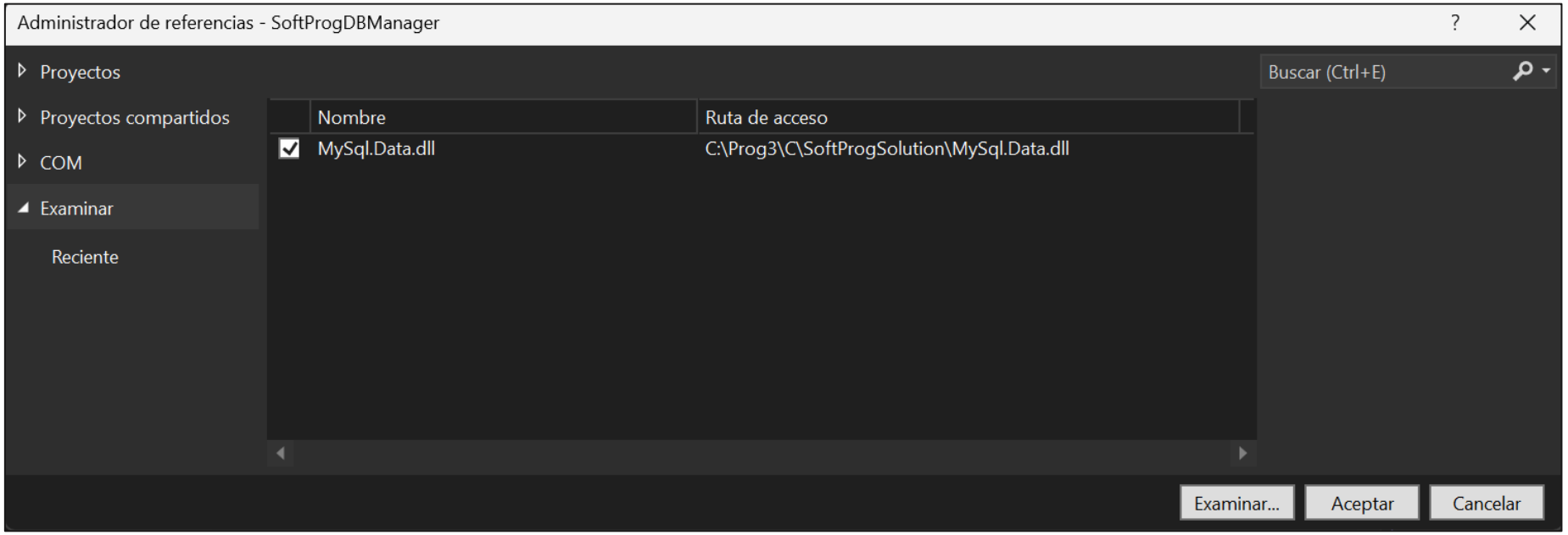
- Colocamos el archivo MySql.Data.dll dentro de la carpeta de la solución.



Nombre	Fecha de modificación	Tipo	Tamaño
SoftProgDBManager	23/04/2026 16:10	Carpeta de archivos	
SoftProgDomain	23/04/2026 14:43	Carpeta de archivos	
SoftProgSolution.slnx	23/04/2026 14:49	Visual Studio Solution	1 KB
MySql.Data.dll	7/04/2026 19:40	Extensión de la aplica...	1,219 KB

Instalación de librerías en C# - Manera Manual

- Dentro del proyecto donde queremos agregar la referencia, damos clic derecho a
  - "Depedencias" -> "Agregar referencia del Proyecto“
- Luego en "Examinar“, ubicamos el archivo .dll

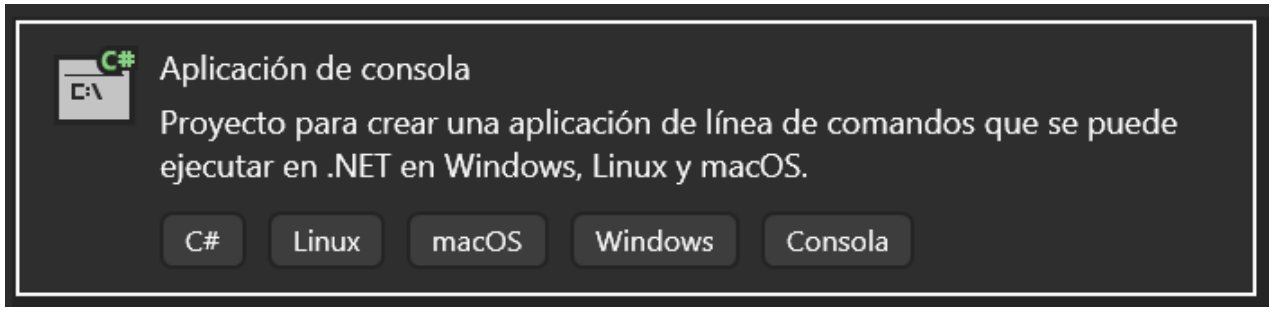


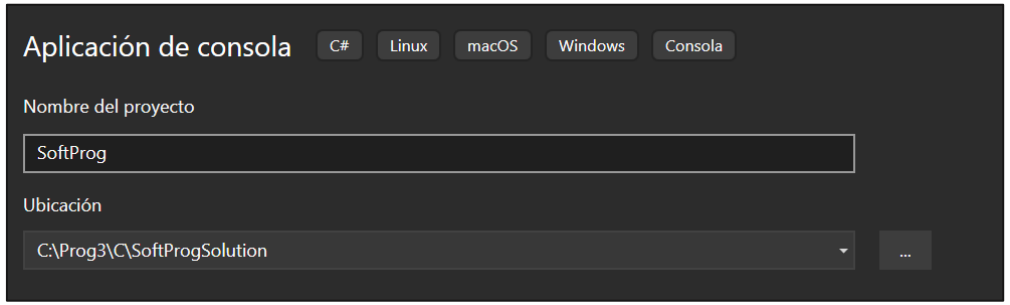
Creación del Proyecto Consola SoftProg

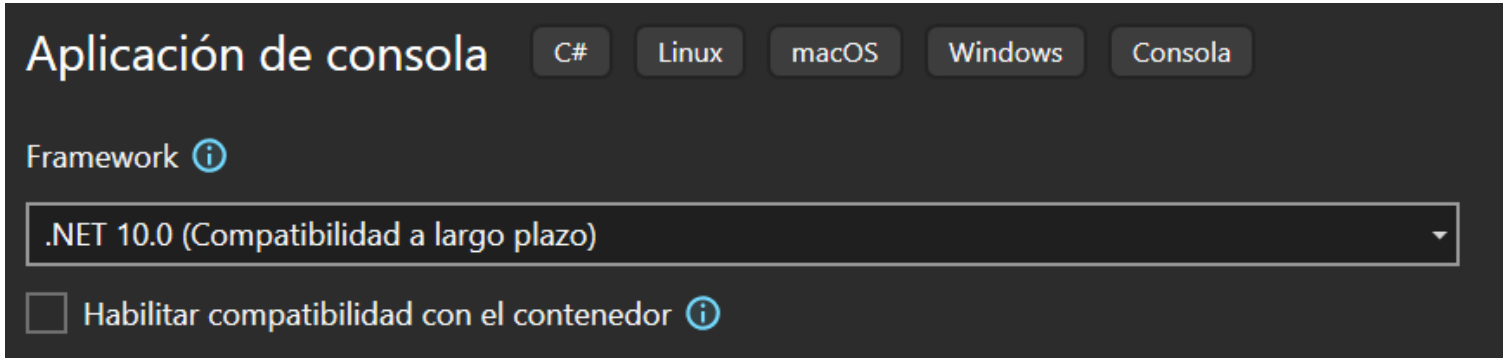
- Para ejecutar y probar la funcionalidad del sistema, se creará un proyecto de tipo aplicación de consola dentro de la solución en Visual Studio.
- Este proyecto se denominará SoftProg y será el encargado de actuar como punto de entrada del sistema, permitiendo la ejecución de las distintas operaciones implementadas.
- A diferencia de las bibliotecas de clases, este tipo de proyecto sí posee un método Main, el cual define el inicio de la ejecución de la aplicación.
- En este proyecto se configurará el archivo appsettings.json, desde donde se obtendrá la cadena de conexión a la base de datos.
- Asimismo, este proyecto hará uso de las clases definidas en otros proyectos, como SoftProgDomain y SoftProgDBManager, mediante referencias entre proyectos.

Creación del Proyecto Consola SoftProg

- Clic derecho a la Solución SoftProgSolution y luego "Agregar", "Nuevo Proyecto".
- Elegimos "Aplicación de consola".

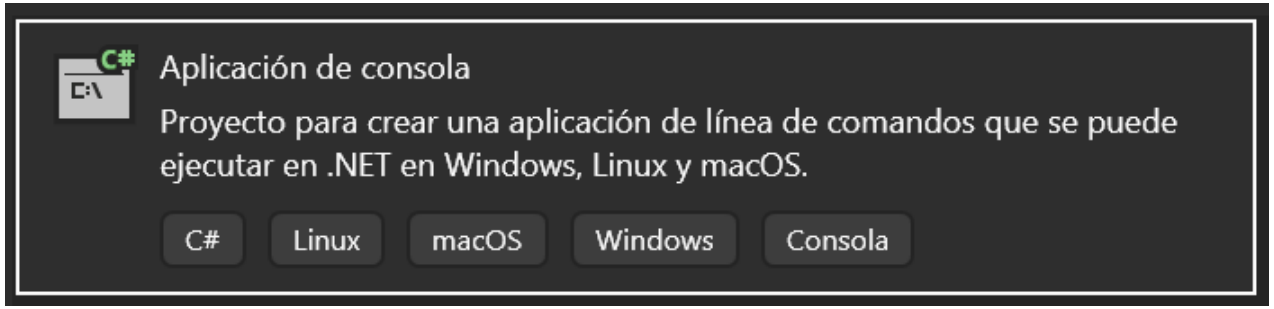


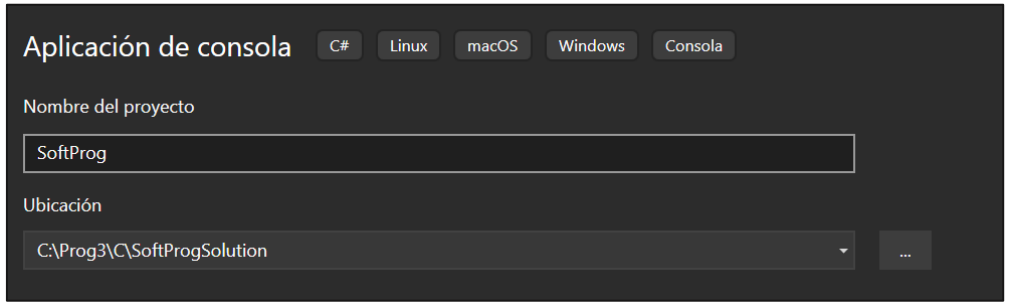


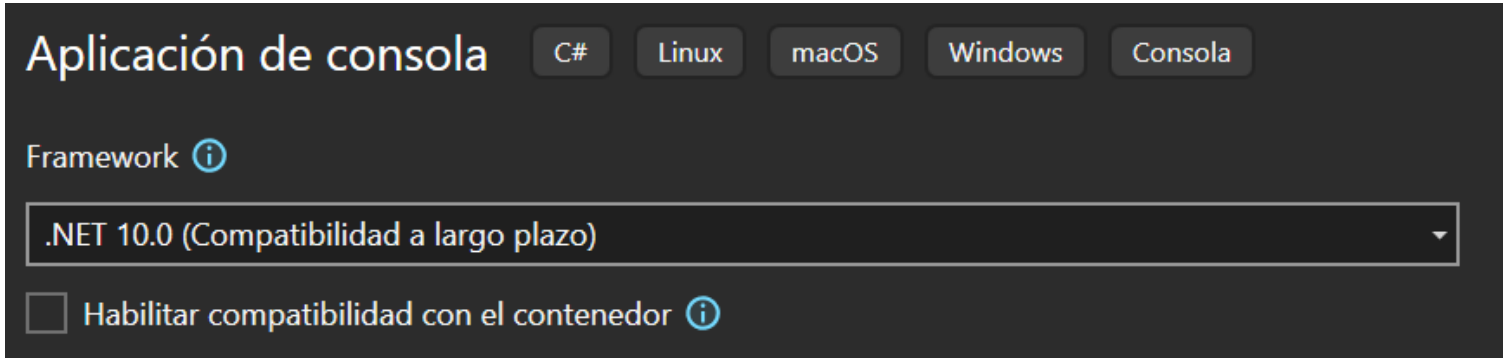


Creación del Proyecto Consola SoftProg

- Clic derecho a la Solución SoftProgSolution y luego "Agregar", "Nuevo Proyecto".
- Elegimos "Aplicación de consola".







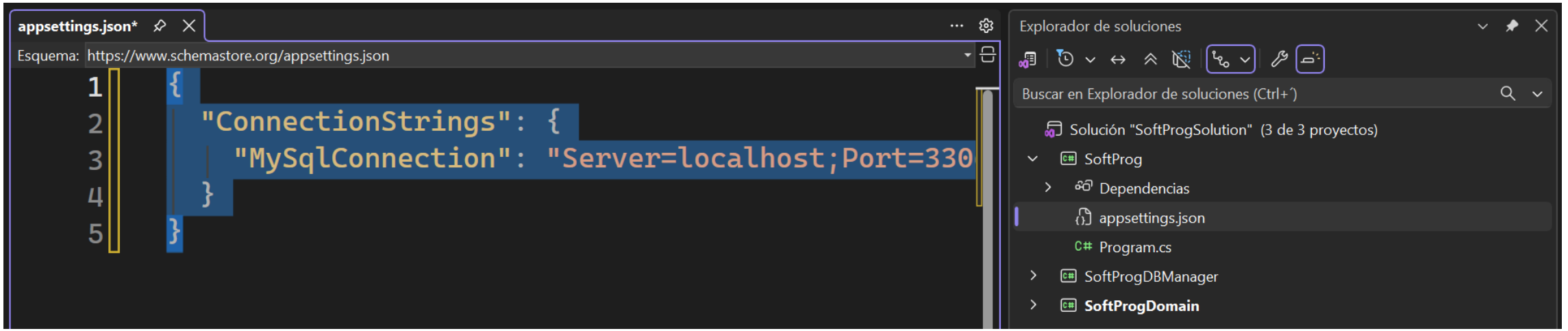
Creación de appsettings.json

- Para gestionar la configuración de la aplicación, se creará un archivo denominado appsettings.json dentro del proyecto de consola SoftProg.
- Este archivo permite almacenar parámetros de configuración de forma estructurada, como la cadena de conexión a la base de datos.
- El formato JSON facilita la organización y lectura de los datos, siendo el estándar en aplicaciones modernas de .NET.

appsettings.json

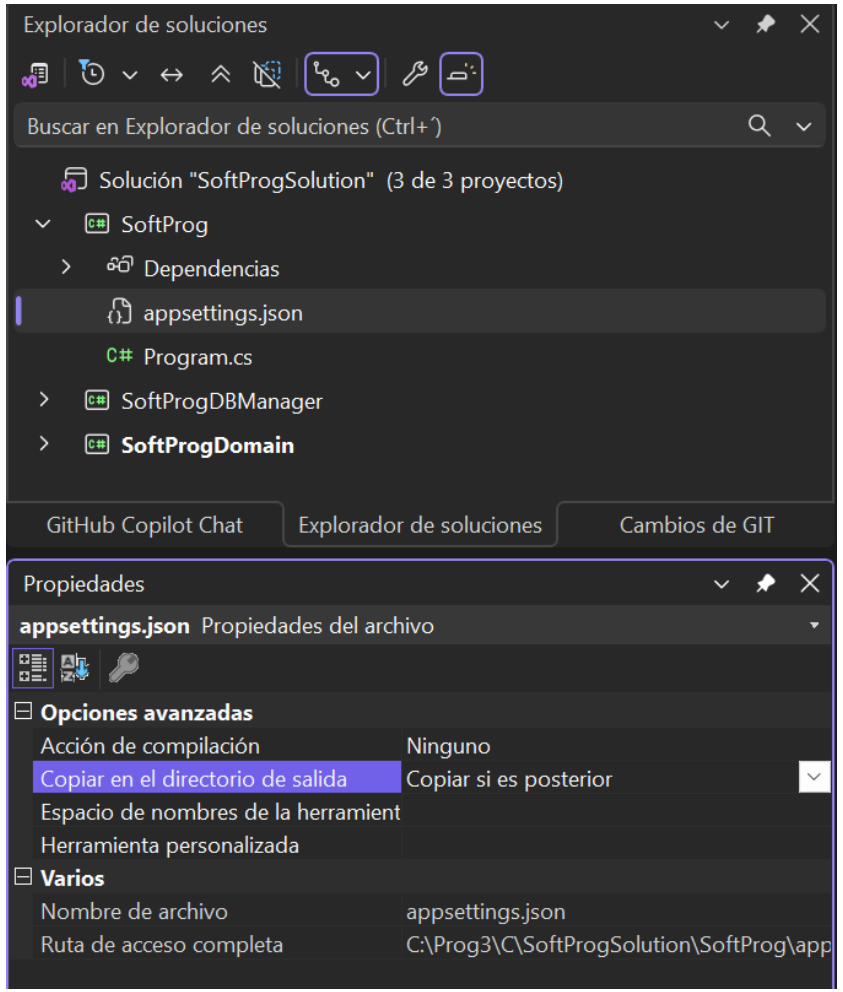
```
{
  "ConnectionStrings": {
    "MySqlConnection": "Server=localhost;Port=3306;Database=softprog;Uid=root;Pwd=1234;"
  }
}
```

Creación de appsettings.json



Creación de appsettings.json

- Es necesario configurar la propiedad “Copiar en el directorio de salida" del archivo como "Copiar si es posterior", para asegurar que esté disponible durante la ejecución de la aplicación.
- Si no copian el archivo al output, el programa no encontrará la configuración.



Referencias entre Proyectos en la Solución

- En una solución organizada en capas, es necesario establecer referencias entre proyectos, de modo que un proyecto pueda utilizar las clases definidas en otro.
- En el contexto de la solución SoftProgSolution, se establecerán las siguientes referencias:
  - SoftProg → SoftProgDBManager
  - SoftProg → SoftProgDomain
  - Para agregar una referencia, se debe hacer clic derecho sobre el proyecto → Agregar → Referencia del Proyecto..., y seleccionar el proyecto correspondiente.
  - Una vez agregada la referencia, es posible utilizar las clases del proyecto referenciado mediante la directiva using.

Referencias entre Proyectos en la Solución

- En una solución organizada en capas, es necesario establecer referencias entre proyectos, de modo que un proyecto pueda utilizar las clases definidas en otro.
- En el contexto de la solución SoftProgSolution, se establecerán las siguientes referencias:
  - SoftProg → SoftProgDBManager
  - SoftProg → SoftProgDomain
  - Para agregar una referencia, se debe hacer clic derecho sobre el proyecto → Agregar → Referencia del Proyecto..., y seleccionar el proyecto correspondiente.
  - Una vez agregada la referencia, es posible utilizar las clases del proyecto referenciado mediante la directiva using.

Inserción de Datos desde el Método Main()

- El método Main inicializa la configuración de la aplicación y establece la conexión a la base de datos utilizando la clase DBManager.
- En este caso, se emplea el método Initialize para configurar la instancia única del DBManager con la cadena de conexión obtenida desde el archivo appsettings.json.
- Posteriormente, se utiliza la propiedad Instance para acceder al DBManager y ejecutar operaciones sobre la base de datos.
- Para insertar datos, se utiliza la clase MySqlCommand junto con parámetros, evitando así problemas de seguridad y mejorando la claridad del código.

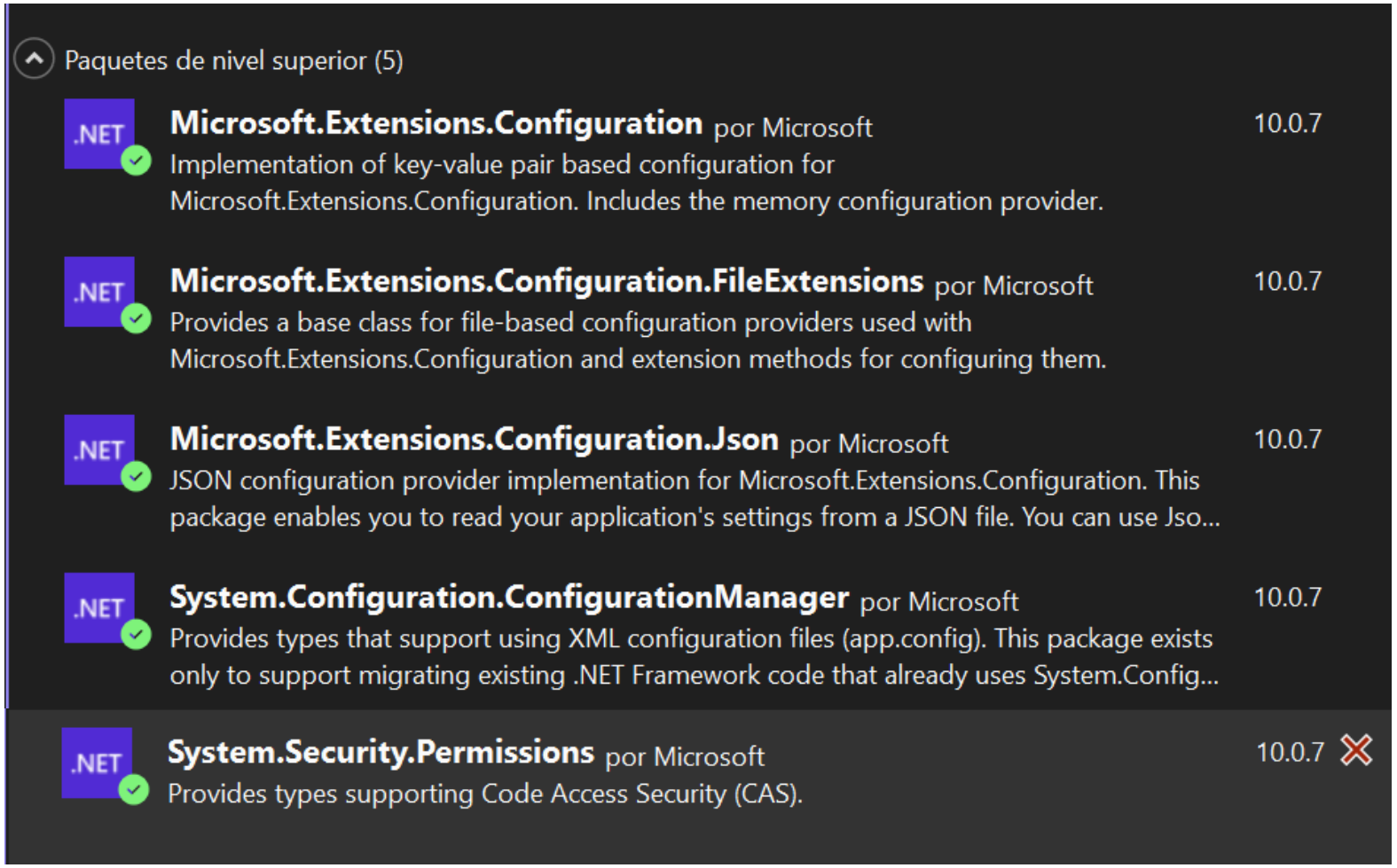
Configuración del Proyecto de Inicio

- En una solución con múltiples proyectos, es necesario definir cuál de ellos será el proyecto de inicio, es decir, el que se ejecutará al iniciar la aplicación.
- En el contexto del sistema SoftProgSolution, el proyecto de inicio debe ser SoftProg, ya que contiene el método Main, que representa el punto de entrada de la aplicación.
- Para configurar el proyecto de inicio en Visual Studio, se debe realizar lo siguiente: Clic derecho sobre el proyecto SoftProg. Seleccionar la opción "Establecer como proyecto de inicio". Una vez configurado, Visual Studio ejecutará automáticamente este proyecto al presionar Run (F5).

Paquetes Requeridos en el Proyecto de Ejecución

- El proyecto de ejecución SoftProg deberá tener instalados los siguientes paquetes para el manejo de configuración y acceso a datos:
  - Microsoft.Extensions.Configuration
  - Microsoft.Extensions.Configuration.Json
  - Microsoft.Extensions.Configuration.FileExtensions
- Asimismo, para el correcto funcionamiento del proveedor de MySQL en .NET moderno, puede ser necesario instalar los siguientes paquetes adicionales:
  - System.Configuration.ConfigurationManager
  - System.Security.Permissions
- Estos paquetes permiten la lectura del archivo appsettings.json y aseguran la compatibilidad de las librerías utilizadas para la conexión a la base de datos.

Paquetes Requeridos en el Proyecto de Ejecución



Inserción de Datos desde el Método Main()

```
using System;
using System.IO;
using Microsoft.Extensions.Configuration;
using MySql.Data.MySqlClient;
using SoftProgDBManager;
class Program
{
    static void Main(string[] args)
    {
        IConfiguration configuration = new ConfigurationBuilder()
            .SetBasePath(Directory.GetCurrentDirectory())
            .AddJsonFile("appsettings.json")
            .Build();
        string connectionString = configuration.GetConnectionString("MySqlConnection");

        // Inicializamos el Singleton
        DBManager.Initialize(connectionString);
```

Inserción de Datos desde el Método Main()

```
        using (MySqlConnection connection = DBManager.Instance.GetConnection())
        {
            connection.Open();

            string sql = "INSERT INTO area(nombre, activa) VALUES (@nombre, @activa)";

            MySqlCommand command = new MySqlCommand(sql, connection);
            command.Parameters.AddWithValue("@nombre", "Recursos Humanos");
            command.Parameters.AddWithValue("@activa", true);

            int filasAfectadas = command.ExecuteNonQuery();

            Console.WriteLine("Filas insertadas: " + filasAfectadas);
        }
    }
}
```

¿Cómo funciona IConfiguration?

- IConfiguration es un objeto que permite leer valores de configuración, como los que están en appsettings.json.
  - ConfigurationBuilder: Es un constructor de configuración. Le indicamos de dónde leer datos.
  - SetBasePath(...): Le indicamos dónde está el archivo, en la carpeta donde corre el programa.
  - AddJsonFile(...): Le indicamos que lea el archivo JSON.
  - Build(): Construye el objeto final (IConfiguration).
- Finalmente, buscamos en el JSON la cadena de conexión llamada MySqlConnection.

¡Gracias!

